

JavaScript fetch() Method

Student-Friendly Complete Notes

1. What is fetch()?

fetch() is a JavaScript method used to communicate with a server or API. It is commonly used to get, send, update, and delete data.

```
fetch("https://dummyjson.com/products")
```

2. Why do we need fetch()?

Suppose a website wants to display users stored on a server. JavaScript uses **fetch()** to request that data from the API.

```
JavaScript → fetch() → API/Server → Response → JavaScript → HTML
```

3. fetch() is Asynchronous

fetch() does not immediately return the final data. It returns a **Promise** because communication with a server takes time.

```
const result = fetch("https://dummyjson.com/users");  
console.log(result); // Promise { <pending> }
```

4. Basic fetch() Syntax

The simplest syntax is:

```
fetch(url)
```

5. Understanding the First Promise

The first **.then()** receives a **Response object**. It contains information such as status, headers, URL, and the response body.

```
fetch("https://dummyjson.com/users")  
  .then((response) => {  
    console.log(response);  
  });
```

6. Why Do We Need the Second Promise?

This is one of the most important concepts. **fetch()** returns a Promise that gives a Response object. Then **response.json()** reads/parses the response body and also returns a Promise.

```
fetch()  
  ↓ Promise  
Response object  
  ↓ response.json()  
  ↓ Promise  
JavaScript data
```

7. Complete Basic Example

This is the common Promise-chaining pattern:

```
fetch("https://dummyjson.com/users")  
  .then((response) => {
```

```

        return response.json();
    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.log(error);
    });

```

8. response.json()

`response.json()` reads the response body and parses JSON into a JavaScript value. After parsing, we can access properties such as `data.name`.

```

const data = await response.json();
console.log(data.name);

```

9. GET Request

GET is generally used to retrieve data. GET is the default method of `fetch()`.

```

fetch("https://dummyjson.com/products")
    .then(response => response.json())
    .then(data => console.log(data));

```

10. POST Request

POST is generally used to send or create data. When sending JSON, use the Content-Type header and `JSON.stringify()`.

```

fetch("https://dummyjson.com/products/add", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        title: "New Laptop",
        price: 50000
    })
})
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));

```

11. PUT Request

PUT is generally used to update an existing resource.

```

fetch("https://dummyjson.com/products/1", {
    method: "PUT",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        title: "Updated Laptop"
    })
})
    .then(response => response.json())
    .then(data => console.log(data));

```

12. PATCH Request

PATCH is generally used to partially update an existing resource.

```

fetch("https://dummyjson.com/products/1", {

```

```

    method: "PATCH",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      price: 45000
    })
  })
  .then(response => response.json())
  .then(data => console.log(data));

```

13. DELETE Request

DELETE is used to delete a resource.

```

fetch("https://dummyjson.com/products/1", {
  method: "DELETE"
})
  .then(response => response.json())
  .then(data => console.log(data));

```

14. fetch() Options

The second argument of `fetch()` is an options/configuration object.

```

fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "John"
  })
});

```

15. JSON.stringify() vs response.json()

JSON.stringify(): JavaScript object → JSON string. Commonly used when sending data.

response.json(): Response JSON → JavaScript value. Commonly used when receiving data.

Sending: Object → `JSON.stringify()` → JSON
 Receiving: JSON response → `response.json()` → JavaScript data

16. Error Handling

Use `.catch()` for rejected promises. However, `fetch()` does not automatically reject just because the server returns HTTP 404 or 500. Check `response.ok`.

```

fetch(url)
  .then(response => {
    if (!response.ok) {
      throw new Error("HTTP Error: " + response.status);
    }
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.log(error));

```

17. response.ok and response.status

response.ok is true for successful HTTP status codes (generally 200–299). **response.status** gives the numeric HTTP status code.

```

console.log(response.ok);
console.log(response.status); // e.g. 200, 404, 500

```

18. Async/Await Version

The same operation can be written using async/await, which many developers find easier to read.

```
async function getUsers() {  
  const response = await fetch(url);  
  const data = await response.json();  
  console.log(data);  
}  
  
getUsers();
```

19. Error Handling with Async/Await

try/catch can be used to handle errors in an async function.

```
async function getUsers() {  
  try {  
    const response = await fetch(url);  
  
    if (!response.ok) {  
      throw new Error("Something went wrong");  
    }  
  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
getUsers();
```

20. Fetch Flow — Most Important

Students should remember this flow:

```
fetch(url)  
↓  
Request sent  
↓  
Promise  
↓  
Response object  
↓  
response.json()  
↓  
Promise  
↓  
JavaScript data  
↓  
Use the data
```

21. Real-Life Example

Imagine ordering food online: you place an order, the request reaches the restaurant, the restaurant responds, and you receive the order information. `fetch()` follows a similar request → response process.

You → Request → Server → Response → Data

22. Common Student Mistakes

Mistake 1: Treating `fetch()` as if it directly returns data.

Mistake 2: Expecting the first `.then()` to contain parsed JSON.

Mistake 3: Forgetting that `response.json()` is asynchronous.

Mistake 4: Sending a JavaScript object directly as a JSON body instead of using `JSON.stringify()`.

```
// Wrong idea:
const data = fetch(url);

// Correct:
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data));
```

23. Complete Example for Students

A practical example combining `fetch()`, `await`, `response.ok`, `json()`, and `try/catch`:

```
async function getProducts() {
  try {
    const response = await fetch(
      "https://dummyjson.com/products"
    );

    if (!response.ok) {
      throw new Error("Failed to fetch products");
    }

    const data = await response.json();

    console.log(data.products);
  } catch (error) {
    console.log(error);
  }
}

getProducts();
```

24. Quick Revision — 7 Points

1. `fetch()` is used to communicate with APIs/servers.
2. `fetch()` returns a Promise.
3. The first `.then()` receives a Response object.
4. `response.json()` also returns a Promise.
5. The second `.then()` receives the parsed data.
6. `JSON.stringify()` is commonly used when sending JSON data.
7. `fetch()` does not automatically reject for HTTP 404/500; check `response.ok`.

```
// Promise chaining
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));

// Async/Await
async function getData() {
  try {
    const response = await fetch(url);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```